

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS

JAVAES

Analizador Léxico para un Lenguaje Personalizado

JAVAES es un lenguaje académico inspirado en Java con palabras clave en español, diseñado para practicar el análisis léxico y sintáctico con ANTLR4 en un front-end de compilador.

Materia	Programación de Sistemas de Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor	Muñoz Quintero Dante Adolfo
Repositorio	https://github.com/Falzo186/JavaES.git
Fecha	Mayo 2026

Nombre	Matrícula
Jesus Alejandro Murillo Zavala	2223330180
Adela Vega Ruiz	2223339021
Clemente Pecina Jose Eduardo	2193217022
Saldaña Nieto Pedro Daniel	2223330192

Tabla de Contenido

INDICE

Tabla de Contenido	2
1. Introducción	3
2. Objetivos	4
2.1 Objetivo General.....	4
2.2 Objetivos Específicos	4
3. Especificación del Lenguaje	5
3.1 Nombre y Propósito	5
3.2 Ejemplos Ilustrativos de Código	5
3.3 Catálogo de Tokens	6
4. Implementación	7
4.1 Definición de Tokens y Palabras Reservadas — JavaESLexer.g4	7
4.2 Reglas Léxicas Adicionales y Estado de Desarrollo	8
4.3 Lógica del Analizador Léxico — LexerRunner.java	9
4.4 Manejo de Errores y Punto de Entrada	10
5. Demostración Funcional	11
5.1 Ejecución con Programa Válido	11
5.2 Ejecución con Errores Léxicos	12
5.3 Caso con Identificadores Inválidos.....	13
6. Conclusiones	14
7. Referencias	15

1. Introducción

El análisis léxico constituye la primera fase de cualquier compilador o intérprete: recibe una secuencia de caracteres como entrada y produce una secuencia de tokens que representan las unidades mínimas con significado del lenguaje. Dominar esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee construir herramientas de procesamiento de lenguajes.

JVAES es un lenguaje académico inspirado en Java cuyas palabras clave están escritas en español. El nombre refleja su propósito principal: servir como banco de práctica para el análisis léxico y sintáctico mediante ANTLR4, herramienta estándar en el desarrollo de front-ends de compiladores.

El lexer fue implementado en Java usando ANTLR4, con una gramática léxica definida en **JavaESLexer.g4** y una gramática sintáctica en **JavaESParser.g4**. **El analizador sintáctico (parser) se encuentra actualmente en desarrollo**; la funcionalidad completamente operativa en esta entrega es el análisis léxico. El sistema transforma archivos fuente *.javaes* en una tabla numerada de tokens (tipo, lexema, línea, columna) e informa con precisión la localización de cualquier error léxico detectado.

Este documento se organiza de la siguiente manera: la Sección 2 presenta los objetivos; la Sección 3 describe la especificación del lenguaje JVAES; la Sección 4 contiene el código fuente de los módulos principales; la Sección 5 muestra capturas de ejecución; las Secciones 6 y 7 presentan conclusiones y referencias.

2. Objetivos

2.1 Objetivo General

Diseñar un lenguaje personalizado denominado **JAVAES** e implementar un analizador léxico funcional en Java con ANTLR4 que transforme programas fuente en listas de tokens, con manejo preciso de errores léxicos que incluya número de línea y columna. El analizador sintáctico se encuentra en una fase inicial de desarrollo.

2.2 Objetivos Específicos

1. Definir la especificación informal del lenguaje JAVAES mediante ejemplos ilustrativos de código que cubran sus construcciones principales.
2. Establecer un catálogo completo de tokens: palabras reservadas en español (tipos, control de flujo, excepciones, clases, modificadores, E/S), identificadores, números (entero, hexadecimal, octal, decimal, científico), operadores y delimitadores.
3. Implementar el analizador léxico usando ANTLR4 con gramática .g4, asegurando reconocimiento correcto de todos los tokens y detección de identificadores inválidos mediante la regla IDENTIFICADOR_INVALIDO.
4. Implementar la detección y reporte de errores léxicos con número de línea, columna y descripción clara del problema mediante LexerErrorListener.
5. Probar el lexer con programas válidos e inválidos que cubran los casos descritos en la especificación.
6. Iniciar el desarrollo del analizador sintáctico (parser) con la gramática JavaESParser.g4 como trabajo en progreso.

3. Especificación del Lenguaje

3.1 Nombre y Propósito

JAVAES (Java en Español) es un lenguaje imperativo de propósito educativo inspirado en Java, cuyas palabras reservadas han sido sustituidas por equivalentes en español. Está diseñado para que estudiantes hispanohablantes comprendan la estructura de un compilador sin la barrera del idioma en las palabras clave. La extensión de los archivos fuente es **.javaes**.

3.2 Ejemplos Ilustrativos de Código

Ejemplo 1 — Clase y método:

```
class Calculadora {  
    entero sumar(entero a, entero b) {  
        retornar a + b;  
    }  
}
```

Ejemplo 2 — Condicional si/sino:

```
entero x = 10;  
si (x > 5) {  
    imprimir("mayor");  
} sino {  
    imprimir("menor o igual");  
}
```

Ejemplo 3 — Ciclo mientras:

```
entero i = 0;  
mientras (i < 10) {  
    i = i + 1;  
}  
retornar i;
```

3.3 Catálogo de Tokens

Categoría	Ejemplos / Tokens	Descripción
KEYWORD — Tipos	ENTERO_TIPO, DECIMAL_TIPO, CADENA_TIPO, BOOLEANO_TIPO, VACIO_TIPO, CARACTER_TIPO	Tipos de dato del lenguaje
KEYWORD — Control	SI, SINO, MIENTRAS, PARA, HACER, CAMBIAR, CASO, DEFECTO, ROMPER, CONTINUAR, RETORNAR	Palabras clave de control de flujo
KEYWORD — Excepciones	INTENTAR, CAPTURAR, FINALMENTE, LANZAR	Manejo de excepciones
KEYWORD — Clases	CLASE, INTERFAZ, NUEVO, ESTE, SUPER	Orientación a objetos
KEYWORD — Modificadores	PUBLICO, PRIVADO, PROTEGIDO, ESTATICO, FINAL	Modificadores de acceso
KEYWORD — Valores	VERDADERO, FALSO, NULO	Literales booleanos y nulo
KEYWORD — E/S	IMPRIMIR, LEER	Entrada y salida estándar
IDENTIFICADOR	x, total, mi_var, resultado2	Letra/_ seguida de letras, dígitos o _; soporta unicode (acentos, ñ)
IDENTIFICADOR_INVALIDO	2b, 123var	Empieza con dígito seguido de letra — genera error léxico específico
ENTERO / HEX / OCTAL	42, 0xFF, 017	Enteros en base 10, 16 y 8
DECIMAL / CIENTÍFICO	3.14, 1.5e10	Decimales y notación científica
CADENA / CARACTER	"hola", 'a'	Literales de texto con soporte de secuencias de escape
OPERADORES	++ -- + - * / % == != >= <= > < && ! += -= *= /= %= =	Aritméticos, relacionales, lógicos y de asignación compuesta
DELIMITADORES	;, () { } [] . : :: -> @ ? & ^ ~	Separadores, agrupadores y operadores especiales
COMMENT	// línea, /* bloque */	Comentarios — descartados (skip)
ERROR_LEXICO	\$, #	Cualquier carácter no reconocido por las reglas anteriores

Nota sobre IDENTIFICADOR_INVALIDO: A diferencia de ERROR_LEXICO (que captura cualquier carácter desconocido), esta regla detecta específicamente secuencias que comienzan con dígito seguido de una o más letras, permitiendo al lexer emitir un mensaje de error más descriptivo e informativo para el usuario.

4. Implementación

A continuación se presenta el código fuente de los módulos principales del analizador léxico. El código completo está disponible en el repositorio GitHub indicado en la portada. El proyecto compila con `compilar.bat` y se ejecuta con `ejecutar.bat`.

4.1 Definición de Tokens y Palabras Reservadas — JavaESLexer.g4

Define la gramática léxica completa de JAVAES mediante ANTLR4. Se organiza en secciones comentadas para mayor legibilidad. El orden de las reglas es crítico: los operadores de dos caracteres (como `==` o `>=`) deben preceder a los de un carácter para evitar tokenización errónea. De igual forma, `DECIMAL_CIENTIFICO` precede a `DECIMAL`, y este a `ENTERO`. La regla `IDENTIFICADOR_INVALIDO`, colocada antes de `ENTERO`, captura específicamente secuencias que comienzan con dígito seguido de letra para generar mensajes de error más descriptivos.

```
lexer grammar JavaESLexer;

// — COMENTARIOS Y ESPACIOS EN BLANCO —————
LINE_COMMENT      : '//' ~[\r\n]* -> skip;
BLOCK_COMMENT     : '/*' .*? '*/' -> skip;
WS                : [ \t\r\n]+ -> skip;

// — PALABRAS RESERVADAS - TIPOS —————
ENTERO_TIPO       : 'entero';   DECIMAL_TIPO : 'decimal';
CADENA_TIPO       : 'cadena';   BOOLEANO_TIPO : 'booleano';
VACIO_TIPO        : 'vacio';    CARACTER_TIPO : 'caracter';

// — PALABRAS RESERVADAS - CONTROL DE FLUJO —————
SI:'si'; SINO:'sino'; MIENTRAS:'mientras'; PARA:'para';
HACER:'hacer'; CAMBIAR:'cambiar'; CASO:'caso'; DEFECTO:'defecto';
ROMPER:'romper'; CONTINUAR:'continuar'; RETORNAR:'retornar';

// — PALABRAS RESERVADAS - EXCEPCIONES —————
INTENTAR:'intentar'; CAPTURAR:'capturar';
FINALMENTE:'finalmente'; LANZAR:'lanzar';

// — PALABRAS RESERVADAS - CLASES Y OBJETOS —————
CLASE:'clase'; INTERFAZ:'interfaz'; NUEVO:'nuevo';
ESTE:'este'; SUPER:'super';

// — PALABRAS RESERVADAS - MODIFICADORES —————
PUBLICO:'publico'; PRIVADO:'privado'; PROTEGIDO:'protegido';
ESTATICO:'estatico'; FINAL:'final';

// — PALABRAS RESERVADAS - VALORES ESPECIALES —————
VERDADERO:'verdadero'; FALSO:'falso'; NULO:'nulo';

// — PALABRAS RESERVADAS - IMPORTACIÓN Y E/S —————
IMPORTAR:'importar'; PAQUETE:'paquete';
IMPRIMIR:'imprimir'; LEER:'leer';

// — LITERALES - CADENAS Y CARACTERES —————
fragment ESC : '\\' . ;
CADENA      : '"' ( ~["\\r\n] | ESC )* '"';
CARACTER    : '\\' ( ~["\\r\n] | ESC ) '\\';

// — LITERALES - NÚMEROS (orden obligatorio) —————
DECIMAL_CIENTIFICO : [0-9]+ '.' [0-9]+ ('e'|'E') ('+'|'-')? [0-9]+;
DECIMAL           : [0-9]+ '.' [0-9]+;
// Identificador inválido: comienza con dígito seguido de letra
IDENTIFICADOR_INVALIDO : [0-9]+ [a-zA-Z_\u00C0-\u017F]
```

```

                                [a-zA-Z0-9_\u00C0-\u017F]*;
ENTERO      : [0-9]+;
ENTERO_HEX  : '0x' [0-9a-fA-F]+;
ENTERO_OCTAL : '0' [0-7]+;

// — IDENTIFICADORES (soporta unicode: acentos, ñ) —————
IDENTIFICADOR : [a-zA-Z_\u00C0-\u017F][a-zA-Z0-9_\u00C0-\u017F]*;

// — OPERADORES ARITMÉTICOS —————
INCREMENTO:'++; DECREMENTO:'--;
SUMA:'+'; RESTA:'-'; MULTIPLICACION:'*'; DIVISION:'/'; MODULO:'%';

// — OPERADORES RELACIONALES (multicarácter primero) —————
IGUALDAD:'=='; DESIGUALDAD:'!=';
MAYOR_IGUAL_QUE:'>='; MENOR_IGUAL_QUE:'<=';
MAYOR_QUE:'>'; MENOR_QUE:'<';

// — OPERADORES LÓGICOS —————
Y_LOGICO:'&&'; O_LOGICO:'||'; NEGACION:'!';

// — OPERADORES DE ASIGNACIÓN COMPUESTA —————
SUMA_ASIGNACION:'+='; RESTA_ASIGNACION:'-=';
MULTIPLICACION_ASIGNACION:'*='; DIVISION_ASIGNACION:'/=';
MODULO_ASIGNACION:'%='; ASIGNACION:'=';

// — DELIMITADORES —————
PARENTESIS_ABIERTO:'('; PARENTESIS_CERRADO:')';
LLAVE_ABIERTA:'{' ; LLAVE_CERRADA:'}';
CORCHETE_ABIERTO:'['; CORCHETE_CERRADO:']';
DOS_PUNTOS_DOBLE:'::'; FLECHA:'->';
PUNTO_Y_COMA:','; COMA:','; PUNTO:'.'; DOS_PUNTOS:':'; ARROBA:'@';

// — OPERADORES ESPECIALES —————
INTERROGACION:'?'; AMPERSAND:'&'; PIPE:'|';
CIRCUNFLEJO:'^'; TILDE:'~';

// — ERROR LÉXICO — captura cualquier símbolo no reconocido —————
ERROR_LEXICO : . ;

```

4.2 Reglas Léxicas Adicionales y Estado de Desarrollo

⚠ En desarrollo: El analizador sintáctico (JavaESParser.g4) se encuentra actualmente en desarrollo. La gramática presentada a continuación define la estructura base del parser, pero su integración completa con recuperación de errores sintácticos, generación de AST y análisis semántico está pendiente para una iteración futura del proyecto.

La gramática sintáctica JavaESParser.g4 establece la jerarquía de producciones desde el símbolo inicial programa hasta las expresiones atómicas. La precedencia de operadores se codifica en la profundidad de las reglas: suma encapsula a producto, que encapsula a atom.

```

parser grammar JavaESParser;
options { tokenVocab=JavaESLexer; }

// Regla inicial
programa: declaracionGlobal* EOF;

declaracionGlobal: claseDecl | metodoDecl | varGlobalDecl;

claseDecl: CLASE IDENTIFICADOR
          LLAVE_ABIERTA miembroClase* LLAVE_CERRADA;

```



```
metodoDecl: tipo IDENTIFICADOR
           PARENTESIS_ABIERTO parametros? PARENTESIS_CERRADO bloque;

tipo: ENTERO_TIPO | DECIMAL_TIPO | CADENA_TIPO
     | BOOLEANO_TIPO | VACIO_TIPO;

// Expresiones con jerarquía de precedencia
expresion : asignacion;
asignacion : comparacion (ASIGNACION asignacion)?;
comparacion: suma ((IGUALDAD|DESIGUALDAD|MAYOR_QUE|MENOR_QUE
                  |MAYOR_IGUAL_QUE|MENOR_IGUAL_QUE) suma)*;
suma      : producto ((SUMA | RESTA) producto)*;
producto  : atom ((MULTIPLICACION | DIVISION | MODULO) atom)*;
atom: PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO
     | ENTERO | DECIMAL | CADENA | VERDADERO | FALSO
     | IDENTIFICADOR;
```

4.3 Lógica del Analizador Léxico — LexerRunner.java

La clase LexerRunner es el núcleo funcional del proyecto. Lee el archivo fuente .javaes, instancia el lexer generado por ANTLR4, elimina el listener de errores por defecto (que imprime a stderr sin formato) y adjunta el LexerErrorListener personalizado. Itera sobre todos los tokens y los presenta en una tabla formateada. Los tokens ERROR_LEXICO e IDENTIFICADOR_INVALIDO generan mensajes de error diferenciados: el primero reporta un carácter no reconocido, el segundo indica explícitamente que el identificador comienza con dígito.

```
public class LexerRunner {
    public static void ejecutarLexico(String rutaArchivo) {
        try {
            String texto = new String(Files.readAllBytes(Paths.get(rutaArchivo)));
            JavaESLexer lexer = new JavaESLexer(CharStreams.fromString(texto));
            lexer.removeErrorListeners();
            lexer.addErrorListener(new LexerErrorListener());
            CommonTokenStream tokens = new CommonTokenStream(lexer);
            tokens.fill();

            // Imprimir encabezado de tabla
            System.out.println("ANÁLISIS LÉXICO - TOKENS DETECTADOS");
            System.out.println("TOKEN | LEXEMA | LÍNEA | COLUMNA");

            boolean hayError = false;
            StringBuilder errores = new StringBuilder();

            for (Token t : tokens.getTokens()) {
                if (t.getType() == -1) break; // EOF
                String nombre = JavaESLexer.VOCABULARY.getSymbolicName(t.getType());
                String lexema = t.getText();
                int linea = t.getLine();
                int col = t.getCharPositionInLine() + 1;

                // Detectar error léxico o identificador inválido
                if (t.getType() == JavaESLexer.ERROR_LEXICO ||
                    t.getType() == JavaESLexer.IDENTIFICADOR_INVALIDO) {
                    hayError = true;
                    String msg = t.getType() == JavaESLexer.IDENTIFICADOR_INVALIDO
                        ? String.format("Identificador inválido '%s' L%d,C%d",
                                         lexema, linea, col)
                        : String.format("Carácter inválido '%s' L%d,C%d",
                                         lexema, linea, col);
                    errores.append(msg).append("\n");
                }
            }
            System.out.printf("%-24s | %-16s | %6d | %6d\n",

```

```

        nombre, lexema, linea, col);
    }
    // Mostrar resumen de errores si los hay
    if (hayError) System.err.println(errores.toString());

    } catch (IOException ex) {
        System.err.println("Error leyendo archivo: " + ex.getMessage());
    }
}
}

```

4.4 Manejo de Errores y Punto de Entrada

LexerErrorListener extiende BaseErrorListener de ANTLR4 para interceptar errores léxicos en tiempo de ejecución y presentarlos con formato legible incluyendo línea y columna exactas.

```

public class LexerErrorListener extends BaseErrorListener {
    @Override
    public void syntaxError(Recognizer,? recognizer,
        Object offendingSymbol, int line, int col,
        String msg, RecognitionException e) {
        System.err.printf(
            "Error léxico en línea %d, columna %d: %s\n",
            line, col, msg);
    }
}

```

Main.java provee el menú interactivo de consola. Lista dinámicamente los archivos .javaes disponibles en resources/examples/Validos/ y resources/examples/Invalidos/, permitiendo seleccionar y analizar cualquiera sin reiniciar el programa.

```

public static void main(String[] args) throws IOException {
    Scanner sc = new Scanner(System.in);
    while (true) {
        System.out.println("JAVAES - Analizador Sintáctico");
        System.out.println("1. Ejecutar análisis léxico");
        System.out.println("2. Ejecutar análisis sintáctico");
        System.out.println("3. Mostrar árbol sintáctico");
        System.out.println("4. Ejecutar ejemplo válido");
        System.out.println("5. Ejecutar ejemplo con error");
        System.out.println("6. Salir");
        String opcion = sc.nextLine().trim();
        switch (opcion) {
            case "1": LexerRunner.ejecutarLexico(resolverRuta(sc)); break;
            case "2": ParserRunner.ejecutarParser(resolverRuta(sc)); break;
            case "4": mostrarSubmen("Validos", sc); break;
            case "5": mostrarSubmen("Invalidos", sc); break;
            case "6": return;
        }
    }
}

```

5. Demostración Funcional

Las siguientes capturas muestran la ejecución del lexer desde la terminal. Archivos de prueba en `resources/examples/Validos/` y `resources/examples/Invalidos/`.

5.1 Ejecución con Programa Válido

Se selecciona la opción **4** del menú (Ejecutar ejemplo válido) y el archivo **ejemplo_basico.javaes**. El programa contiene declaraciones de variables enteras y una expresión aritmética. No presenta errores léxicos.

```
JAVAES - Analizador Sintáctico
=====
1. Ejecutar análisis léxico
2. Ejecutar análisis sintáctico
3. Mostrar árbol sintáctico
4. Ejecutar ejemplo válido
5. Ejecutar ejemplo con error
6. Salir
Seleccione una opción: 4

=====
Ejemplos Validos
=====
1. ejemplo_basico.javaes
2. ejemplo_clase.javaes
3. ejemplo_control_flujo.javaes
4. ejemplo_expandido.javaes
5. ejemplo_if.javaes
6. ejemplo_metodo.javaes
7. ejemplo_while.javaes
0. Volver al menú principal
Seleccione un ejemplo: 1
```

Captura 5.1a — Menú principal y selección del ejemplo válido `ejemplo_basico.javaes`.

ANÁLISIS LÉXICO - TOKENS DETECTADOS			
TOKEN	LEXEMA	LÍNEA	COLUMNA
ENTERO_TIPO	entero	1	1
IDENTIFICADOR	x	1	8
ASIGNACION	=	1	10
ENTERO	10	1	12
PUNTO_Y_COMA	;	1	14
ENTERO_TIPO	entero	2	1
IDENTIFICADOR	y	2	8
ASIGNACION	=	2	10
ENTERO	20	2	12
PUNTO_Y_COMA	;	2	14
ENTERO_TIPO	entero	3	1
IDENTIFICADOR	z	3	8
ASIGNACION	=	3	10
IDENTIFICADOR	x	3	12
SUMA	+	3	14
IDENTIFICADOR	y	3	16
PUNTO_Y_COMA	;	3	17

Captura 5.1b — Tabla de tokens generada para `ejemplo_basico.javaes` (sin errores léxicos).

5.2 Ejecución con Errores Léxicos

Se selecciona la opción **5** y el archivo **error_lexico.javaes**. El archivo contiene el símbolo \$, no reconocido por la gramática. El lexer lo registra como **ERROR_LEXICO** en la tabla y emite el mensaje con línea y columna en el panel inferior.

```

=====
JAVAES - Analizador Sintáctico
=====
1. Ejecutar análisis léxico
2. Ejecutar análisis sintáctico
3. Mostrar árbol sintáctico
4. Ejecutar ejemplo válido
5. Ejecutar ejemplo con error
6. Salir
Seleccione una opción: 5

=====
Ejemplos Invalidos
=====
1. error_lexico.javaes
2. error_lexico.operador.javaes
3. error_sintactico.javaes
0. Volver al menú principal
Seleccione un ejemplo: 1

--- Ejemplo: error_lexico.javaes ---

ANÁLISIS LÉXICO - TOKENS DETECTADOS

+-----+
| TOKEN | LEXEMA | LÍNEA | COLUMNA |
+-----+
| ENTERO_TIPO | entero | 1 | 1 |
| ERROR_LEXICO | $ | 1 | 8 |
| IDENTIFICADOR | x | 1 | 9 |
| ASIGNACION | = | 1 | 11 |
| ENTERO | 10 | 1 | 13 |
| PUNTO_Y_COMA | ; | 1 | 15 |
+-----+

ERROR LÉXICO DETECTADO

Error léxico: carácter inválido '$' en línea 1, columna 8.

```

Captura 5.2 — Detección de símbolo inválido '\$' en línea 1, columna 8. El resto del programa se tokeniza correctamente.

5.3 Caso con Identificadores Inválidos

Se ejecuta **error_identificador.javaes**. Contiene el identificador `2b` que comienza con dígito. La regla **IDENTIFICADOR_INVALIDO** lo captura y emite el mensaje: *"Error léxico: identificador inválido '2b' en línea 2, columna 12"*. El resto de los tokens válidos continúa procesándose sin interrupciones.

```
=====
Ejemplos Invalidos
=====
1. error_identificador.javaes
2. error_lexico.javaes
3. error_lexico operador.javaes
4. error_sintactico.javaes
0. Volver al menú principal
Seleccione un ejemplo: 1

--- Ejemplo: error_identificador.javaes ---

ANÁLISIS LÉXICO - TOKENS DETECTADOS

+-----+
| TOKEN          | LEXEMA      | LÍNEA | COLUMNA |
+-----+
| ENTERO_TIPO    | entero      | 1      | 1        |
| IDENTIFICADOR  | procesar    | 1      | 8        |
| PARENTESIS_ABIERTO | (          | 1      | 16       |
| ENTERO_TIPO    | entero      | 1      | 17       |
| IDENTIFICADOR  | a           | 1      | 24       |
| PARENTESIS_CERRADO | )          | 1      | 25       |
| LLAVE_ABIERTA  | {           | 1      | 27       |
| ENTERO_TIPO    | entero      | 2      | 5        |
| IDENTIFICADOR_INVALIDO | 2b        | 2      | 12       |
| ASIGNACION     | =           | 2      | 15       |
| ENTERO         | 5           | 2      | 17       |
| PUNTO_Y_COMA   | ;           | 2      | 18       |
| RETORNAR       | retornar    | 3      | 5        |
| IDENTIFICADOR  | a           | 3      | 14       |
| SUMA           | +           | 3      | 16       |
| IDENTIFICADOR  | b           | 3      | 18       |
| PUNTO_Y_COMA   | ;           | 3      | 19       |
| LLAVE_CERRADA  | }           | 4      | 1        |
+-----+

ERROR LÉXICO DETECTADO

Error léxico: identificador inválido '2b' en línea 2, columna 12.
```

Captura 5.3 — Identificador inválido '2b' detectado en línea 2, columna 12. El lexer continúa procesando el resto del archivo.

6. Conclusiones

Jesus Alejandro Murillo Zavala. Diseñar la gramática léxica en ANTLR4 me permitió entender en la práctica por qué el orden de las reglas es crítico: cuando coloqué ENTERO antes que DECIMAL_CIENTIFICO, el lexer tokenizaba 1.5e10 en fragmentos separados. Verlo fallar y corregirlo fue la forma más efectiva de interiorizar el concepto de 'longest match' en los lexers.

Adela Vega Ruiz. El mayor reto fue el manejo de errores léxicos sin detener el análisis del resto del programa. Implementar el mecanismo de recuperación —marcar el token como ERROR_LEXICO y continuar— requirió entender el flujo interno de ANTLR4. El resultado fue un módulo de errores mucho más útil para el usuario final.

Clemente Pecina Jose Eduardo. Separar la gramática léxica (JavaESLexer.g4) de la gramática sintáctica (JavaESParser.g4) facilitó enormemente las pruebas: podía modificar una expresión regular y ejecutar inmediatamente los casos de prueba sin tocar el parser. Esa separación de responsabilidades es algo que aplicaré en proyectos futuros.

Saldaña Nieto Pedro Daniel. Implementar la detección del error de 'identificador que comienza con dígito' requirió distinguir entre un NUMBER_INT legítimo y uno seguido inmediatamente de letras. La solución fue la regla IDENTIFICADOR_INVALIDO antes de ENTERO en el .g4, lo cual me hizo comprender mejor la precedencia de reglas en ANTLR4.

7. Referencias

- [1] Parr, T. (2013). The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.
- [2] ANTLR Project. (2024). ANTLR 4 Documentation. Recuperado de <https://www.antlr.org/>
- [3] Oracle Corporation. (2024). Java SE 21 Documentation. Recuperado de <https://docs.oracle.com/en/java/javase/21/>